

C++&Qt Day03 hw1 알고리즘 보고서

작성자: Doyeon Park

작성일: 2026.09.04

목차

1. 알고리즘이란?
2. 시간 복잡도
3. 공간 복잡도
4. 트리
5. 그래프
6. Dijkstra 알고리즘
7. A* 알고리즘

1. 알고리즘이란?

- 어떤 입력을 받아서, 정해진 순서대로 처리해서, 원하는 출력을 만들어내는 방법

1.1 알고리즘의 명확성

- 알고리즘은 각 단계가 애매하면 안된다.

1.2 알고리즘의 유한성

- 알고리즘은 언젠가는 끝나야한다.

while (true) { }

처럼 영원히 도는 것만 있다면 원하는 결과에 도달하는 알고리즘이라고 하기 어려움.

2. 시간 복잡도

- 데이터가 많아질수록 프로그램이 얼마나 더 많은 일을 해야하는지를 나타냄

- 보통 입력 크기 n 이 증가할 때 연산 횟수가 얼마나 증가하는가를 봄

2.1 $O(1)$

ex) 배열 한 위치 접근

-예를 들어

```
int arr[5] = {10, 20, 30, 40, 50};
```

```
cout << arr[3];
```

이 코드는 바로 `arr[3]`으로 가기 때문에 배열 크기가 5 든 10000 이든 `arr[3]`을 찾는 데 필요한 작업은 거의 동일하다.

따라서 $O(1)$ 즉, 상수 시간이라 부른다.

2.2 $O(n)$

ex)선형 탐색

- 예를 들어 배열에서 숫자 7 을 찾는다고 가정하자.

```
int arr[5] = {2, 4, 6, 7, 9};
```

```
for (int i = 0; i < 5; i++)  
{
```

```

    if (arr[i] == 7)
    {
        cout << "찾음";
    }
}

```

이 방식은 배열의 앞에서부터 순차적으로 찾기 때문에 운이 나쁘면 데이터가 n 개일때 n 개를 전부 확인해야 할수도 있다.

따라서 선형 탐색은 $O(n)$ 이다.

2.3 $O(n^2)$

ex) 이중 반복문

- 예를 들어

```

for (int i = 0; i < n; i++)
{
    for (int j = 0; j < n; j++)
    {
        cout << i << " " << j << endl;
    }
}

```

위 코드 같은 경우 바깥 반복문이 n 번 돌고, 그 안에서 또 n 번이 돈다.

따라서 $n \times n = n^2$ 이기 때문에 이중 반복문은 시간복잡도가 $O(n^2)$ 가 된다.

2.4 $O(\log n)$

ex) 이진 탐색

예를들어 정렬된 숫자 **1 2 3 4 5 6 7 8** 이 있다 가정하자.

여기서 **7** 을 찾는다 할 때, **7** 은 **4** 보다 크기때문에 왼쪽은 버리게 된다.

1 2 3 [4] 5 6 7 8 → **5 6 7 8**

또 가운데를 보고 **7** 은 **6** 보다 크므로 절반을 버린다.

5 [6] 7 8 → **7 8**

이런식으로 자료 8 개로 예시를 들었지만 데이터가 1000 개가 있어도 10 번정도만 비교하면 되는 탐색 방법이다.

2.5 $O(n \log n)$

ex) 병합 정렬

$O(n \log n)$ 은 시간복잡도가 $O(n)$ 보다 크고 $O(n^2)$ 보다 작은 알고리즘으로 보통 효율적인 정렬 알고리즘에서 많이 나오는 복잡도이다.

2.6 시간복잡도 정리

복잡도	PDF 예시	느낌
$O(1)$	배열 한 위치 접근	거의 일정
$O(\log n)$	이진 탐색	매우 빠름
$O(n)$	선형 탐색	데이터 수만큼
$O(n \log n)$	병합 정렬	정렬에서 자주 나옴
$O(n^2)$	이중 반복문	데이터 커지면 급격히 느려짐

3. 공간 복잡도

- 공간 복잡도는 입력 크기가 커질 때 메모리를 얼마나 더 쓰냐를 본다.

예를 들어

```
int a = 10;
```

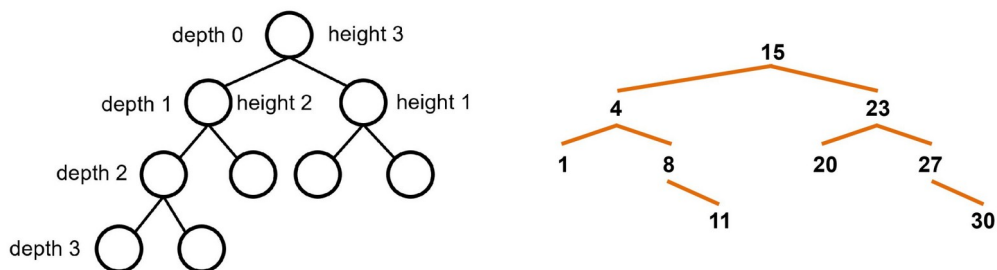
처럼 변수 하나만 쓰면 입력 크기와 관계없이 메모리가 거의 일정한 반면,

```
int* arr = new int[n];
```

처럼 n 개짜리 배열을 만들면 n 이 커질수록 메모리도 같이 증가하므로 `int a = 10;`와 차이가 있다.

4. 트리(Tree)

트리는 데이터를 계층 구조로 표현하는 자료구조로 형태는 다음과 같다.



맨 위에 하나가 있고, 그 아래로 가지처럼 계속 뻗어나가는 것을 볼 수 있다.

4.1 트리의 기본 용어

4.1.1 노드(Node)와 간선(Edge)

오른쪽 그림에서 볼 수 있는 **15, 4, 23** ... 이런것들이 **노드(Node)**이다.

노드와 노드를 연결하는 선을 **간선(Edge)**라고 한다.

4.1.2 루트 노드(Root Node) / 리프 노드 (Leaf Node)

트리에서 가장 위에있는 노드를 **루트 노드**라고 한다.

위 그림 기준으로 숫자 **15** 가 **루트 노드**라고 할 수 있다.

트리에서 자식이 없는 노드를 **리프 노드**라고 한다.

위 그림을 기준으로 **1 11 20 30** 이 **리프 노드**이다.

4.1.3 부모 노드 / 자식 노드

위 그림을 기준으로 **15, 4, 23** 이 세 숫자를 보자.

여기서 **15** 는 **4** 와 **23** 의 **부모 노드**이고, **4, 23** 은 **15** 의 **자식 노드**이다.

이 관계는 상대적으로 작용한다.

위 그림에서 **1, 4, 15** 이 세 숫자를 보면 **4** 는 **1** 의 **부모 노드**이자 **15** 의 **자식 노드**가 된다.

4.1.4 Depth / Height

Depth 는 루트 노드에서 해당 노드까지의 경로에 있는 간선 개수이다.

위 사진을 보면 **아래로 내려갈수록 Depth 도 증가**하는 것을 볼 수 있다.

Height 은 해당 노드에서 리프 노드까지 가장 긴 경로의 간선 개수이다.

예를 들어 위 사진에서 숫자 **15** 는 **Height 3**, 숫자 **8** 은 **Height 1** 임을 알 수 있다.

4.1.5 트리의 높이

트리의 높이는 루트에서 가장 깊은 리프까지의 거리이다.

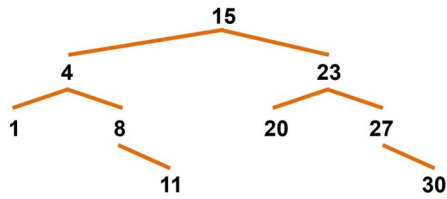
위 두 사진 모두 가장 깊은 리프까지 간선이 3 개이므로 높이는 3 이다.

4.2 이진 탐색 트리

이진 탐색 트리는 데이터를 넣을 때부터 정렬된 구조로 저장한다.

일반적으로 이진 탐색 트리는 다음 규칙을 따른다.

왼쪽 자식 노드 값 < 부모 노드 값 < 오른쪽 자식 노드 값



(이진 탐색 트리)

이진 탐색 트리의 장점 :

예를 들어 숫자 20 을 찾는다고 가정하자.

루트가 15 이므로 왼쪽은 볼 필요가 없어진다.

15 → 23 → 20

오른쪽으로 가면 23 이 나오고 20 은 23 보다 작으므로 왼쪽에 있는 노드를 탐색한다.

이렇게 데이터를 다 볼 필요 없이 일부만 보고 효율적으로 찾을 수 있다는 장점이 있다.

이진 탐색 트리의 단점:

다만 데이터 입력 순서에 따라 성능이 달라진다는 단점이 있다.

예를 들어

15, 4, 23 , 8, 1, 20, 11, 27, 30 순서대로 데이터를 넣으면 위와같은 사진의 트리가 나오게 된다.

이런경우 탐색이 빠르다.

하지만,

1, 4, 8, 11, 15, 20, 23, 27, 30 처럼 이미 정렬된 값을 순서대로 넣으면 1 이 루트 노드가 되면서 사실상 연결리스트처럼 되어버린다.

따라서 만약 30 을 찾으려 할때 모든 데이터를 봐야 할 수도 있다는 단점이 생긴다.

(최악의 경우 시간복잡도 $O(n)$)

4.3 기타 트리

4.3.1 AVL 트리

트리가 한쪽으로 너무 기울어지면 자동으로 구조를 조정해서 균형을 맞추는 트리이다.

따라서 탐색 성능이 $O(\log n)$ 수준을 유지하기 쉽다.

4.3.2 Red-Black Tree

이 트리도 심하게 기울지 않도록 규칙을 두고 균형을 유지하는 트리이다.

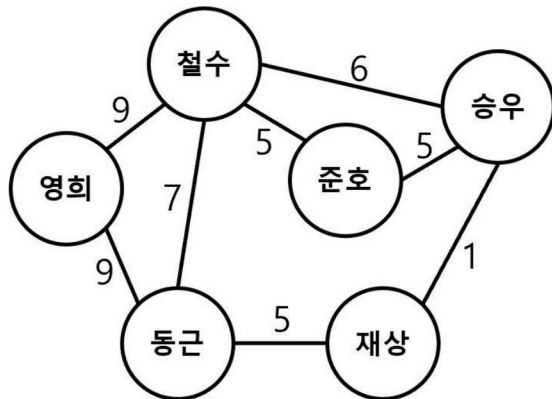
4.3.3 KD Tree

(50, 50), (10, 70), ... 이런 데이터들을 공간적으로 나눠서 저장하는 트리이다.

좌표, 위치 같은 데이터를 다룰 때 유용하다.

5. 그래프(Graph)

그래프는 여러 개의 점과, 그 점들을 잇는 연결 관계를 표현하는 자료구조이다.



그래프에는 Vertex(정점), Edge(간선), Weight(가중치)이 있다.

Vertex:

그래프의 점 하나하나를 정점이라 부른다.

Edge:

정점과 정점을 연결하는 선을 의미한다.

Weight:

간선에 붙는 값이다.

가중치(Weight)의 예시로 거리, 시간, 비용, 위험도 등이 있다.

그래프는 무향그래프와 유향그래프가 존재한다.

무향그래프:

방향이 없는 그래프이다.

A ----- B

이렇게 연결되어있으면

A → B

B → A

둘다 가능하다.

즉, 길이 양방향이라고 보면 된다.

예를 들면 사람이 걸어다니는 일반 도로 중 양방향 도로같은 경우가 있다.

유향그래프:

유향 그래프에는 간선에 방향이 있다.

$A \rightarrow B$

이렇게 연결되어 있으면

$A \rightarrow B$ 는 가능하지만,

$B \rightarrow A$ 는 무조건 가능한 것은 아니다.

일방통행이라 볼 수 있다.

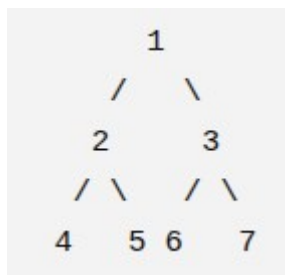
5.1 그래프 탐색

그래프를 만들었다면 이제 그래프를 어떤 순서로 탐색할 것인지 문제가 생긴다.

여기서 BFS, DFS 탐색을 알아보자

BFS:

BFS는 Breadth-First Search의 약자로 한국어로는 너비 우선 탐색이라고 부른다.

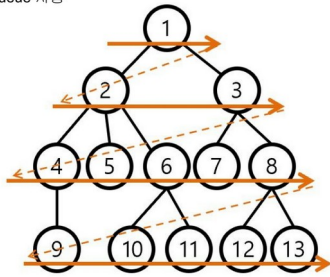


이런 그래프가 있다 가정하자.

이걸 BFS로 탐색하면

$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7$ 순서로 탐색하게 된다.

가까운 노드부터 탐색
Queue 사용



(너비 우선 탐색 과정)

BFS 는 주로 Queue 를 사용한다.

Queue 는 먼저 들어온게 먼저 나가는 구조로 되어있다.

예를 들어

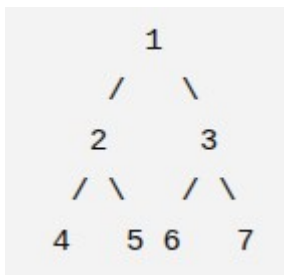
$A \rightarrow B \rightarrow C$ 순서로 들어왔다면

$A \rightarrow B \rightarrow C$ 순서로 나가게 된다.

따라서 BFS 는 주변 노드를 순서대로 저장해두고, 먼저 발견한 노드부터 탐색하기에 Queue 가 잘 맞는다.

DFS:

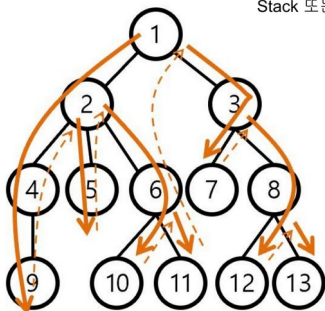
DFS 는 Depth-First Search 의 약자로 한국어로는 깊이 우선 탐색이라 부른다.



같은 트리를 보자

DFS 는 $1 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 3 \rightarrow 6 \rightarrow 7$ 의 순서로 탐색하게 된다.

한 방향으로 최대한 깊게 탐색
Stack 또는 재귀 사용



(깊이 우선 탐색 과정)

DFS 는 Stack 을 이용하여 구현할 수 있다.

Stack 은 데이터를 $A \rightarrow B \rightarrow C$ 순서로 쌓았을 때,

꺼낼 때는 $C \rightarrow B \rightarrow A$ 순서로 꺼내게 된다.

DFS 는 한 방향으로 계속 깊게 들어가야 하므로 Stack 이 잘 맞는다.

6. Dijkstra 알고리즘

현재까지 가장 적은 비용으로 갈 수 있는 곳부터 하나씩 확정하면서 목적지까지 최단 경로를 찾는 알고리즘이다.

6.1 비용이란?

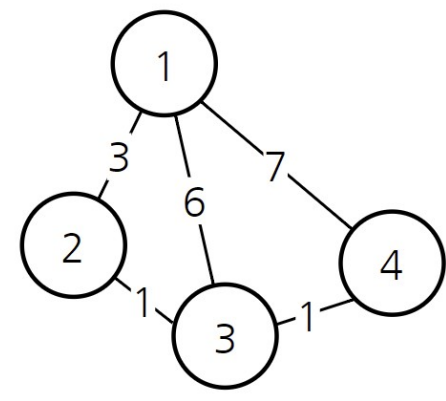
비용은 그래프에서 언급했었던 가중치(Weight)를 의미한다.

예를 들어 $A -3-- B -2-- C$ 라고 하면

$A \rightarrow B$ 비용은 3 이고, $B \rightarrow C$ 비용은 2 이므로 $A \rightarrow C$ 까지 누적 비용은 $3 + 2 = 5$ 가 된다.

다익스트라는 이 누적 비용을 계속 비교하는 알고리즘이다.

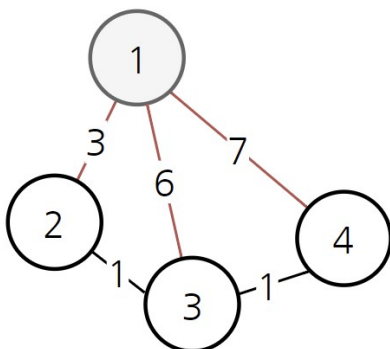
6.2 예시



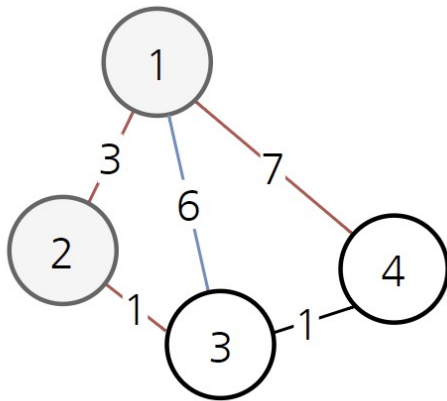
위 그래프 상에서 1 에서 3 으로 간다고 가정하자.

일단 기본적으로 1 에 바로 옆에 붙어있는 노드인 2, 3, 4 까지의 최단거리를 각각 3, 6, 7 로 산정할 수 있다.

현재까지는 $1 \rightarrow 3$ 으로 가는 최소비용은 6 이다.



1 → 4 로 가는동안 가중치가 7 이기 때문에 이미 6 을 초과했으므로 저 경로는 탈락하게 된다.



이후 다음 순서에서 1→3 의 비용이 6 인데에 반해 경로 1 → 2 → 3 의 비용은 4 로 더 저렴하다는 계산이 나오면 1 에서 3 으로 가는 최소비용을 4 로 갱신하게 된다.

정리하면 다음과 같다.

1. 출발 노드 설정
2. 출발 노드를 기준으로 각 노드의 최소 비용 저장
3. 방문하지 않은 노드 중에서 가장 비용이 적은 노드 선택
4. 해당 노드를 거쳐서 특정한 노드로 가는 경우를 고려하여 최소 비용 갱신
5. 3~4 번 과정 반복

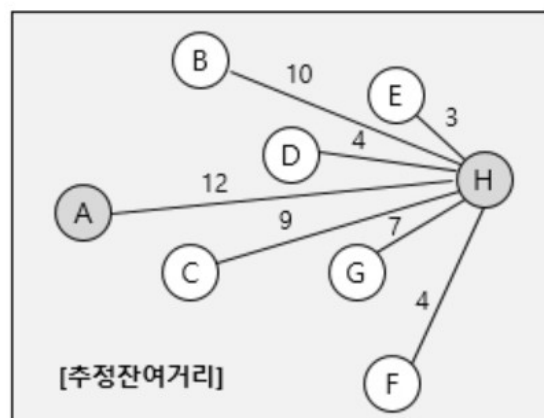
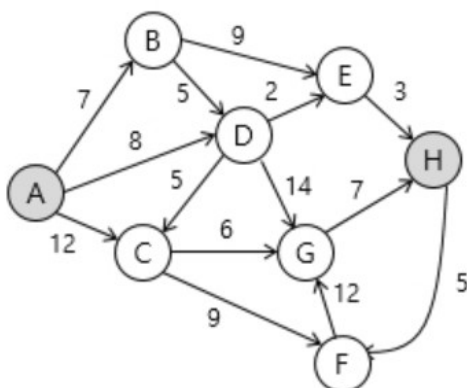
7. A* 알고리즘

이전의 다익스트라 알고리즘은 지금까지 실제로 든 비용이 가장 작은 곳부터 탐색한 알고리즘이었다면 A*는 여기서 하나를 더 본다.

A*는 목적지까지 앞으로 얼마나 남았는지도 같이 보는 알고리즘이다.

7.1 예시

예제를 보며 설명하면



위 사진에서 A 에서 H 까지 최단경로를 찾아서 야한다고 가정하자.

A* 알고리즘은 시작점에서부터 현재 정점까지의 최단거리와 현재 정점에서부터 목표점까지의 추정잔여거리를 합산하여 최종경로를 구하게 되므로 만약 현재 정점이 D 라고 한다면 A->D 의 최단거리와 D-H 의 추정 잔여거리를 합산하게 된다. (단, 추정잔여거리는 목표점과 각 정점을 잇는 직선거리)

시작점에서 각 정점에서 이르는 거리를 $g(x)$,

목적점에서 각 정점에 이르는 추정잔여거리(평가함수)를 $h(x)$ 라 하면, 다음과 같은 식을 도출 할 수 있다.

$$f(x) = g(x) + h(x)$$

이를 이용하여 A 에서 H 까지의 최단 경로를 찾아보자

먼저 A 에서 출발할때 $A \rightarrow B$, $A \rightarrow D$, $A \rightarrow C$ 세가지의 경우의 수가 있다.

$$A \rightarrow B \text{ 인 경우 } f(x) = g(x) + h(x) = 7 + 10 = 17$$

$$A \rightarrow D \text{ 인 경우 } f(x) = g(x) + h(x) = 8 + 4 = \mathbf{12 \text{ (선택)}}$$

$$A \rightarrow C \text{ 인 경우 } f(x) = g(x) + h(x) = 12 + 9 = 21$$

D 를 기준으로 $D \rightarrow E$, $D \rightarrow G$ 두가지의 경우의 수가 있다.

$$D \rightarrow E \text{ 인 경우 } f(x) = g(x) + h(x) = 10 + 3 = \mathbf{13 \text{ (선택)}}$$

$$D \rightarrow G \text{ 인 경우 } f(x) = g(x) + h(x) = 22 + 7 = 29$$

따라서 최단 경로는 $A \rightarrow D \rightarrow E \rightarrow H$ 임을 알 수 있다.